

Git Hub を勉強してみよう

参考

[Git の基本操作を作業を例に学ぶ #GitHub - Qiita](#)

ハードウェアエンジニアをしています、ファイルの変更管理がとてもめんどくさい！

なんか Git や GitHub を使うとめっちゃくちゃ便利らしいので、マジで知識 0 から Git ・

GitHub を勉強してみます。この note はその軌跡です

今回は、Git のバージョン管理の仕組みを学習します。

そもそも Git/GitHub ってなんぞや

バージョン管理システム

端的に言うと、バージョン管理システム。よく会社で見かけるカスみたいな文章保存を例に見てみよう。

- 「レポート_最終版.txt 📄」
- 「レポート_最終版_修正.txt 📄」
- 「レポート_本当の最終版_最新.txt 📄」

こういう「名前だけを変える」という管理をした場合、

「どれが一番新しいのか」

「そもそも何を変えたのか」

「以前の状態に戻せない」

といったトラブルが起こる。

特に複数人でファイルを編集した場合、上書き保存されてしまって誰かの変更を消してしまう、ファイル名のルールを逸脱してぐちゃぐちゃにする(ぶち●すぞ)という事態も発生しやすい。

この問題を解決するために作られたのが **Git** と **GitHub** である。

すばらしい。ぜひ使いたい

それぞれの特徴を見てみよう。

Git

ファイルの変更履歴（いつ・誰が・どこを変えたか）を自動で記録・管理する仕組みである。

いつでも過去の状態に戻したり、過去の変更内容と比較ができたりとかなり便利である

GitHub

Git で記録した履歴データをインターネット上に保存し、チームメンバーと共有・共同編集できるようにした Web サービスである。

なんと無料で使えちゃう！神かな？

Git と GitHub の違い

「履歴管理をしてくれる便利なソフト」が Git であり、

「Git で管理している履歴をクラウドに保存するサービス」が GitHub である。

Git の履歴管理

最も基礎となる 3 要素

Git の履歴管理の最も基礎的な手順は、以下 3 つとなる

1. ワークツリー

ファイルを作成したり、プログラムを書いたりする普段の作業スペース

2. ステージングエリア

「このファイル更新したいよ」を乗せる場所。

3. リポジトリ

「この時点の状態を確定！」として、変更履歴を正式に記録する場所

わかりにくいな、ということで、写真に例えて見てみよう

ワークツリー

散らかった写真フォルダ中。保存したいもの・したくないものがゴロゴロ転がっている場所。

ステージングエリア

アルバム。散らかった写真フォルダ(ワークツリー)から保存したいものだけを持ってくる場所。

リポジトリ

ロッカー。アルバム(ステージングエリアに保存したデータ)を保存しておく場所。同じ名前のアルバムがある場合、古いアルバムを履歴として残して、新しいアルバムを表に出しておける便利なロッカー。ついでに古いアルバムと新しいアルバムのどこが変わったか、どこのどいつが編集したかを一発で見られるすばらしい場所

おお、わかりやすい。でもなんでわざわざワークツリーからリポジトリに直接保存しないのだろうか？いらんものを消してしまえばいいだけのようにも見える。

もし直接保存した場合どうなるか見てみよう。

ステージングエリアという仕組み

「いらない変更を消してから、リポジトリに保存すればいいじゃないか」

たしかに、一人で小さなプログラムを書いているだけなら、それでも良さそうに見える。

では、写真の例で考えてみよう。

ワークツリーにこんな写真が入っていると仮定する。

IMG001.jpg 観光地の写真

IMG002.jpg 観光地の写真

IMG003.jpg ピンボケ

IMG004.jpg 友達との写真

IMG005.jpg 同じ場所を撮った写真

IMG006.jpg 夜景

IMG007.jpg 間違えて撮った床

今日の作業では、「観光地の写真だけをアルバムに保存したい」とする。

Git にステージングエリアという仕組みがなければ、いらんものを消してしまって

IMG001.jpg 観光地の写真

IMG002.jpg 観光地の写真

こんな形にしてリポジトリに保存。一見問題なさそうだ。

しかし、保存した後で「あ、やっぱり IMG004 も保存したかった」となった場合、二度と元に戻せない。削除してしまっているからだ。

保存したいもと、もしかすると使うかもしれないものと、いらんものをごちゃまぜに保存できる場所 → ワークツリー

一旦保存することが確定したものを置く場所 → ステージングエリア

確定したものを正式に保存する場所 → リポジトリ

この3つで管理することで、後々「やっぱ必要だったわ」となってもめんどくさいことになりにくいのだ。

Git 用語

git add

保存したい変更点をステージングエリアに追加すること

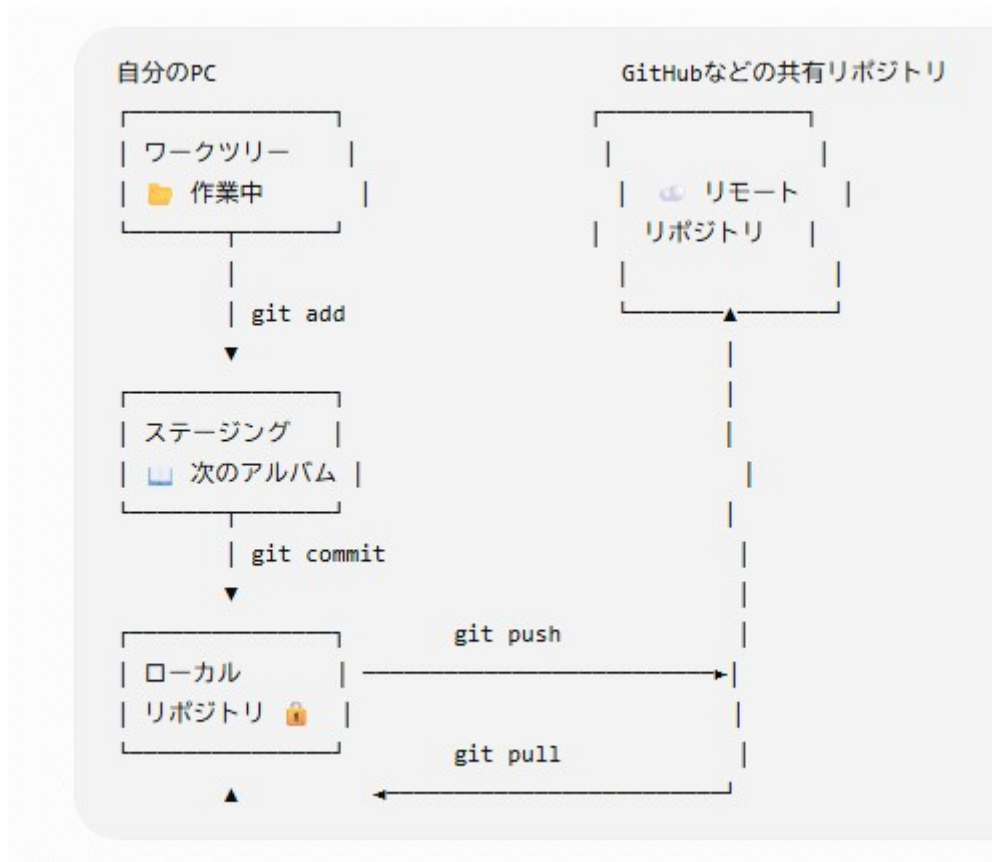
git commit

ステージングエリアに保存したものを、履歴管理に追加すること

git push

git pull

は次回学習する



図で表す

とこんな感じ

実際の流れ

ワークツリー

IMG001.jpg という写真を編集した。

IMG001.jpg を編集

編集内容：

空をきれいにした

顔を明るくした

エフェクトを追加した

しかし、しっくりしなかったので空をきれいにした、という変更だけを保存することにした

↓ **git add**

ステージングエリア

空をきれいにした、という保存だけしたいのでこうなる

IMG001.jpg を編集

編集内容：

空をきれいにした

↓ **git commit**

リポジトリ

commit A

IMG001.jpg ← 元々の写真

↓

commit B

IMG001.jpg ← 今回更新した写真

ステージングエリアで入れなかった内容

顔を明るくした

エフェクトを追加した

という変更は、ワークツリーにそのまま残してあるので、気が向いたらいつでも追加できる。

リポジトリ

リポジトリは**単なるファイル置き場**ではない。

もう一度流れを復習しよう

ワークツリー

↓

写真を編集

↓ add

ステージング 「この状態を保存したい」

↓ commit

リポジトリ 「はい、この瞬間のアルバムを正式に記録します」

先程の流れで、追加で「顔を明るくした」や「エフェクトを追加した」を変更履歴として加える場合、

リポジトリ

■ Commit 1

IMG001.jpg

└─ 空をきれいにした

■ Commit 2

IMG001.jpg

└─ 空をきれいにした

+顔を明るくした

■ Commit 3

IMG001.jpg

└─ 空をきれいにした

+顔を明るくした

+エフェクトを追加した

このように、変更が積み上がっていく。

そしてコミットには、複数の情報を持たせられる。

誰が変更した？

いつ変更した？

何を変更した？

なぜ変更した？

Git での操作

すこし先取りにはなるが、git ではこれらをどのように操作するのか、見てみよう。

前提

こんな写真が保存されている

アルバム 1 ファイルパス：/Users/gotoh/photo/

└── IMG001.jpg 素の写真

└── IMG002.jpg 素の写真

└── IMG003.jpg 素の写真

git init

アルバム 1 というフォルダを、Git 管理することにした。

アルバム 1 の中で、以下のコマンドを入力する

```
git init
```

そうすると、こんな返事が帰ってきた

```
$ git init
```

```
Initialized empty Git repository in /Users/gotoh/photo/.git/
```

意味は、

このフォルダを Git のリポジトリとして使えるようにしました

ということ。.git という隠しフォルダが追加されているが、これが git の管理情報を入れるフォルダとなる。

git status

ファイルの情報を確認するコマンド。

```
git status
```

git init した直後の「アルバム 1」の中で実行すると

```
$ git status
```

On branch main

No commits yet

Untracked files:

(use "git add <file>..." to include in what will be committed)

IMG001.jpg

IMG002.jpg

IMG003.jpg

nothing added to commit but untracked files present (use "git add" to track)

意味はこんな感じ

On branch main

↓

現在 main というブランチにいます

No commits yet

↓

まだ一度も commit していません

Untracked files:

↓

Git がまだ管理対象として認識していないファイルがあります

IMG001.jpg

IMG002.jpg

IMG003.jpg

↓

この 3 枚です

nothing added to commit

↓

commit するためにステージングされたものはありません

git add

git 管理されてないファイルがあるぞと怒られてしまったので、まずはそのまま、

IMG001.jpg IMG002.jpg IMG003.jpg の 3 つをリポジトリに入れてみる。

一旦ステージングエリアに移動させるので

```
git add IMG001.jpg IMG002.jpg IMG003.jpg
```

と入力。ところが

```
$
```

何も表示されない。

残念ながら、Git は「やったよ」とご丁寧に説明してくれる仕様ではない。

ちゃんとステージングエリアに入ったか見たいので、もう一度 `git status` を実行

```
$ git status
```

On branch main

No commits yet

Changes to be committed:

(use "git rm --cached <file>..." to unstage)

```
new file:   IMG001.jpg
```

```
new file:   IMG002.jpg
```

```
new file:   IMG003.jpg
```

ここで見てほしいのがこれ

Changes to be committed:

こいつは、

「次に commit すると、この変更が commit されますよ」

という意味である。すなわち、ステージングエリアに入っている、という状態。